Team Division for Communication & Notifications Module

3-Person Team Split

PERSON 1: Real-Time Messaging Specialist

Focus: Socket.IO + Message System

Responsibilities:

Database Models (2)

- `Conversation` **model**
- `Message` **model**
- **Create migrations for both**

REST APIs

1. POST /api/v1/conversations # Start new conversation
2. GET /api/v1/conversations # List user's conversations
3. GET /api/v1/conversations/:id # Get single conversation
4. POST /api/v1/messages # Send message
5. GET /api/v1/messages/:conversationId # Get message history
6. PATCH /api/v1/messages/:id/read # Mark message as read

Socket.IO Implementation

- **Setup Socket.IO server**

- **JWT authentication for sockets**
- **Handle events:**
  - `join_conversation`
  - `send_message`
  - `message_read`
  - `typing`
  - `stop_typing`
- **Real-time message delivery**
- **Online/offline user status**

Files to Create:

```
src/modules/communication/
├── models/
│   ├── Conversation.ts
│   └── Message.ts
├── controllers/
│   ├── conversation.controller.ts
│   └── message.controller.ts
├── services/
│   ├── conversation.service.ts
│   └── message.service.ts
├── routes/
│   ├── conversation.routes.ts
│   └── message.routes.ts
└── socket/
    └── chat.socket.ts
```

Testing:

- Test messaging between 2 users
- Test read receipts
- Test message persistence
- Test typing indicators

- Test with multiple concurrent users

PERSON 2: Notification System Specialist

Focus: In-App Notifications + Preferences

Responsibilities:

Database Models (2)

- **`Notification` model**
- **`NotificationPreference` model**
- **Create migrations for both**

REST APIs

1. GET /api/v1/notifications # Get user's notifications
2. GET /api/v1/notifications/unread-count # Count unread
3. PATCH /api/v1/notifications/:id/read # Mark single as read
4. PATCH /api/v1/notifications/read-all # Mark all as read
5. DELETE /api/v1/notifications/:id # Delete notification
6. GET /api/v1/notifications/preferences # Get preferences
7. PATCH /api/v1/notifications/preferences # Update preferences

Notification Service

Create notifications for these events:

- New message received
- Job application status changed
- New job posted (matching candidate profile)
- Job approved/rejected by admin
- Application received (recruiter)

Socket.IO Integration

- **Emit `new_notification` event to users**
- **Emit `notification_count_update` event**

Files to Create:

```
src/modules/communication/
├── models/
│   ├── Notification.ts
│   └── NotificationPreference.ts
├── controllers/
│   └── notification.controller.ts
├── services/
│   └── notification.service.ts
├── routes/
│   └── notification.routes.ts
└── utils/
    ├── notificationTypes.ts        # Enum of notification types
    └── notificationFactory.ts      # Create different notifications
```

Testing:

- Create notifications programmatically
- Test notification preferences save/update
- Test marking as read
- Test real-time notification delivery via socket
- Test notification filtering by type

PERSON 3: Email System Specialist

Focus: Email Notifications + Integration

Responsibilities:

Email Service Setup

- Configure SendGrid or Nodemailer
- Setup email templates (HTML + Plain text)
- Create email queue system (optional: Bull/Redis)

Email Templates

Create templates for:

1. New Message - "You have a new message from [Name]"
2. Application Status Changed - "Your application status: [Status]"
3. New Job Alert - "New job matching your profile"
4. Job Approved/Rejected - For recruiters
5. Welcome Email - Account created
6. Email Verification - (if not in Auth module)

Email Trigger Logic

- **Check `NotificationPreference` before sending**
- **Send email only if:**
  - **User has email notifications enabled**
  - **User is offline OR hasn't read in-app notification in 5 minutes**
- **Batch emails where possible (daily digest option)**

Integration Points

Work with Person 2 to trigger emails when notifications are created:

// When notification created:

1. Save to database (Person 2)
2. Emit socket event (Person 2)
3. Check if email should be sent (Person 3)
4. Send email if conditions met (Person 3)

Files to Create:

```
src/modules/communication/
├── services/
│   ├── email.service.ts
│   └── emailQueue.service.ts        # Optional
├── templates/
│   ├── emails/
│   │   ├── newMessage.html
│   │   ├── applicationStatus.html
│   │   ├── jobAlert.html
│   │   ├── jobApproval.html
│   │   └── welcome.html
│   └── emailLayouts/
│       └── base.html                # Common layout
└── utils/
    └── emailHelper.ts               # Format emails, add logo, etc.
```

Testing:

- Test email sending with real SendGrid/SMTP
- Test all email templates render correctly
- Test email preferences are respected
- Test email queuing (if implemented)
- Test emails have correct links and data

Suggested Timeline

Day 1: Setup & Models
-Everyone:Create your database models & migrations
- Everyone: Run migrations, test DB structure
- Team sync: Review models together

Day 2-3: Core Features
- Person 1:Build REST APIs + basic Socket.IO
- Person 2:Build notification REST APIs
- Person 3: Setup email service + create 2 templates

Day 4: Integration
- Person 1 + 2: Connect messaging to notifications
- Person 2 + 3: Connect notifications to emails
- Team sync: Test full flow together

Day 5: Advanced Features
- Person 1:Add typing indicators, online status
- Person 2: Add notification filtering, bulk mark as read
- Person 3: Complete all email templates, add styling

Day 6: Testing & Documentation
- Everyone:Write tests for your code
- Everyone: Document your APIs
- Team sync: Full system test

Shared Resources You All Need
Common Utilities (Create together or assign to Person 2)

src/modules/communication/utils/
├── constants.ts # Notification types, message types
├── validators.ts # Request validation schemas
└── helpers.ts # Common helper functions

Middleware(Person 1 can create these)

src/modules/communication/middlewares/
├── conversationAccess.middleware.ts # Check user owns conversation
└── socketAuth.middleware.ts # Authenticate socket connections

Individual Checklists

Person 1 Checklist:

- ☐ Conversation model created
- ☐ Message model created
- ☐ REST APIs working (6 endpoints)
- ☐ Socket.IO server setup
- ☐ Socket authentication working
- ☐ Real-time messages working
- ☐ Read receipts working
- ☐ Typing indicators working
- ☐ Integration with Person 2's notification service

Person 2 Checklist:

- ☐ Notification model created
- ☐ NotificationPreference model created
- ☐ REST APIs working (7 endpoints)
- ☐ Notification service can create all types
- ☐ Socket integration for real-time notifications
- ☐ Integration with Person 1's message service
- ☐ Integration with Person 3's email service
- ☐ Default preferences set on user registration

Person 3 Checklist:

- ☐ Email service configured (SendGrid/Nodemailer)
- ☐ 5+ email templates created
- ☐ Email templates tested and look good
- ☐ Email preference checking works
- ☐ Integration with Person 2's notification service
- ☐ Emails send successfully
- ☐ Email links work correctly

- [ ] Unsubscribe functionality (optional)